LLM Foundations & Inference Economics

1. The core loop

An LLM is a deep stack of identical blocks. Each block does two things: **self-attention** (every token looks back at the earlier tokens and decides which matter) and a **feed-forward** step (each token is transformed on its own). Stack 60 to 100 of these, train on trillions of tokens to predict the next one, and you have a frontier model.

- The decisive fact: it writes **one token at a time, left to right**, feeding each new token back in
- That loop explains almost everything operational: why output costs more, why long answers are slow, why caching works, why chain-of-thought helps

2. Attention

- Each token computes a **Query** (what am I looking for?), a **Key** (what do I offer?) and a **Value** (what do I contribute if picked)
- Its Query is scored against every earlier Key, the scores become weights that sum to 1, and the output is the weighted sum of Values
- **Quadratic cost**: every token attends to every earlier token, so doubling the context roughly quadruples attention compute

> ⚠ Don't stuff everything into context. 4k relevant tokens from retrieval usually beat a 100k-token dump on both quality and cost.

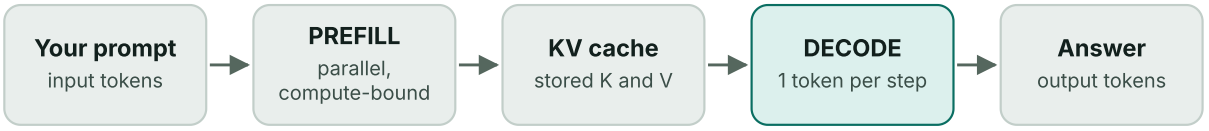
3. Four upgrades to name

- **GQA** (grouped-query attention): groups of query heads share one Key/Value pair, shrinking the KV cache 4 to 8× with little quality loss
- **MoE** (mixture of experts): many expert sub-networks, a router activates only a few per token. **Active** parameters drive cost; **total** parameters drive memory. Never compare models by headline parameter count
- **RoPE**: how word position is injected. Long-context versions are retrained, not switched on. **Lost-in-the-middle**: facts buried mid-context are recalled less reliably, so put critical material at the start or end
- **Reasoning models**: generate hidden thinking tokens before answering. Better on hard problems, but thinking is billed as output and adds latency. Telling them 'think step by step' is redundant

4. The two phases of generation

- **Prefill** (reading your input): one parallel pass that builds the KV cache. Compute-bound, cheap per token, and it sets **TTFT** (time to first token, ~0.5 to 2 s)
- **Decode** (writing the output): one token per step, each step reading the whole KV cache. Memory-bandwidth-bound, sets **TPOT** (~15 to 40 ms per token)
- That asymmetry is why **output costs ~5× input** (e.g. $3 in / $15 out per million tokens)
- KV cache size per token = 2 × layers × KV heads × head dimension × bytes per value. A modern 70B GQA model is about 0.33 MB per token, so ~33 GB at 100k context

✎ DIAGRAM



Prefill is cheap and sets time-to-first-token; decode is sequential, reads the whole KV cache every step, and costs about 5× per token.

5. Tokens and the multilingual tax

- A token is a sub-word chunk from **byte-pair encoding**: roughly 4 characters or 0.75 English words
- Tokenizers are trained mostly on English, so other scripts need more tokens for the same meaning: **Arabic ~1.5 to 2.5×**, French ~1.2 to 1.6×
- Triple hit: higher cost, higher latency (more decode steps) and less effective context
- The professional move: a 10-minute benchmark of tokens per 100 characters on real target-language text, turned into a **per-language budget multiplier**

6. Sampling and the determinism myth

- **Temperature** sharpens or flattens the next-token distribution: 0 to 0.3 for extraction and structured output, 0.5 to 0.8 for chat and RAG, ~1.0 for creative work
- **top_p** keeps only the most likely tokens up to a probability mass p (0.9 to 0.95). Tune temperature or top_p, not both
- Temperature 0 is **not** deterministic: GPU floating-point maths, request batching, MoE routing and provider hardware all vary

- For regulated clients promise **reproducibility of process**: pinned model versions, logged prompts and parameters, archived outputs

7. Why models fail

- **Hallucination**: a confident, fluent, false statement. It's a consequence of predicting plausible text, not true text. **Intrinsic** = contradicts the source you gave it (fix with grounding and faithfulness evals); **extrinsic** = unverifiable from any source (fix with retrieval)
- **Miscalibration**: the model sounds equally sure whether right or wrong, so don't trust its stated confidence
- **Sycophancy**: it agrees with the user because training rewarded agreeable answers
- Hard limits without tools: exact arithmetic, strict counting, long-horizon planning

8. Structured output

- Reliability ladder, best first: **constrained decoding / strict schema** → forced tool call → JSON mode → 'please reply in JSON' (fails 1 to 5% of the time forever)
- Validate every payload on receipt anyway; it turns silent corruption into a loud error

9. The four kinds of caching

- **KV cache**: inside one request, automatic. Not a fix you propose
- **Prompt (prefix) caching**: a provider feature across requests. Reuses the processing of an **identical prompt start**; cached tokens are ~90% cheaper and the first token comes sooner. The model still writes a fresh answer. Put stable content first, variable content last
- **Response cache**: your system stores the full answer for an exact repeat question and skips the model
- **Semantic cache**: returns a stored answer for a *similar* question. Biggest saving, biggest risk; only for stable FAQs with a strict threshold, never for personal or live data

> ⚠ One timestamp or user name at the top of the prompt silently kills the prompt cache.

✎ DIAGRAM

Cache	Lives where	Saves	Risk
KV cache	Inside one call	Recomputing past tokens	None: how inference works
Prompt cache	Provider, across calls	Re-reading a fixed prefix	Misses if the prefix changes
Response cache	Your system	Whole call, exact repeat	Stale answers
Semantic cache	Your system	Whole call, similar question	Wrong answer to a near-miss

Four different things called 'caching'. Mixing them up is an instant credibility leak.

10. Prompt engineering toolkit

- **Zero-shot** with clear structure first; add **few-shot** examples (2 to 5, chosen to cover tricky cases) only if needed
- Say who the output is for and what it's used for; separate instructions, context and the question
- **Chain-of-thought** for non-reasoning models only; **self-consistency** (sample several, take the majority) for accuracy at extra cost
- **Prompt chaining**: split big tasks into small testable calls (extract → classify → draft)
- Version prompts in git, test every change against the golden set, change one thing at a time

11. Cost levers, in order

1. **Cut output tokens** (output is a small share of tokens but close to half the bill)
2. **Cache the stable prefix**
3. **Route by difficulty**: cheap model for easy work, strong model for hard work
4. **Batch API** for anything not interactive (~50% off)

- Prices change monthly: quote only from the provider's live page with an 'as of' date

SAY IT

"Output costs about 5× input because every output token is its own pass over the whole KV cache, so I'd cut output first, then cache the stable prefix, then route easy requests to a cheaper model."